Program Optimization for Verifiable Problems

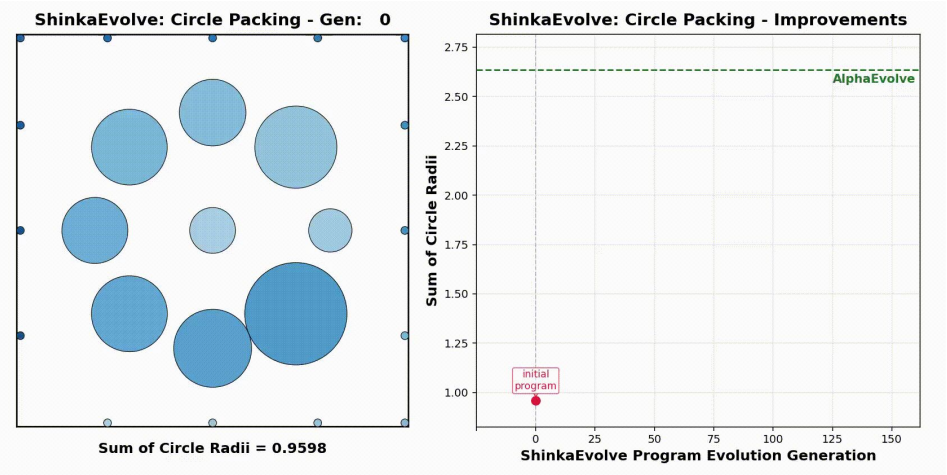
AlphaGrad

Problem Statement and Motivation

- LLMs are good at **generating** solutions, but bad at **verifying** them
  - Many approaches have tried to answer this (ex. LLM as a Judge)
- One solution: evaluate LLMs against a **<u>strictly verifiable</u>** problem

Many such examples:

- Kernel speed
- Matrix multiplication algorithm speed
- Hard geometry problems

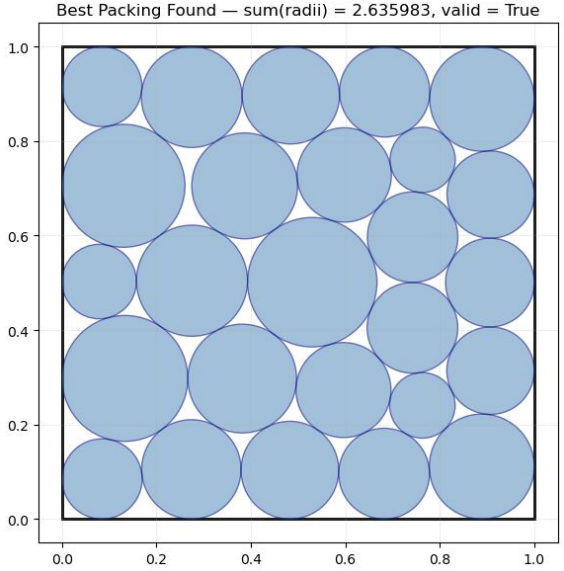


Previously we tried…

- Train open-source model (gpt-oss-120b) with LoRA
- Generate 64 rollouts per batch, then fine-tuned to upweight best solutions
- Save a bank of good solutions, re-use on every next batch

Problems:

- Very compute expensive (cost ~$30 for 5 batches)
- RL is very unstable/hard to find reward
- Programs need to be saved at each iteration



Technical approach

GEPA **optimize_anything**

- A reflection LLM proposes edits to the solver program
- Solver program is executed locally and calls LLM function
- Evaluator runs the candidate program and verifies constraints

Why GEPA?

- Avoids training weights and instead performs structured search over programs
- Improves compute efficiency of finding best solution (can use very small LLMs)
  - Gemini 2.5 Flash Lite, o3-mini, etc.

LLM Prompt: 3 variations

```
result = optimize_anything(
    seed_candidate={"solver_code": SEED_SOLVER},
    evaluator=evaluate,
    objective=(
        f"Optimize a Python program that uses an LLM (OpenAI {LLM_MODEL}) to suggest "
        f"circle packing strategies for n={NUM_CIRCLES} circles in the unit square [0,1]×[0,1]. "
        "The program should maximize the sum of radii while satisfying all constraints. "
        "Output must be valid JSON with format: {\"circles\": [[x,y,r], ...], \"sum_radii\": float}"
    ),
    background=(
        "The solver receives an llm_function(prompt) -> text that it can call. "
        "The solver should craft effective prompts to guide the LLM, parse responses robustly, "
        "and may apply local optimization or validation steps. "
        "CRITICAL: DO NOT SEARCH FOR THE SOLUTION. The best solution found is 2.7320508075688772"
        "\n\nConstraints:"
        "\n- No overlap: distance between any two circle centers >= sum of their radii"
        "\n- Boundaries: for each circle at (x,y) with radius r: 0 <= x-r, x+r <= 1, 0 <= y-r, y+r <= 1"
        "\n- Must output exactly 26 circles"
        "\n\nThe solver code should be a complete Python script that:"
        "\n1. Imports necessary libraries (numpy, json, google.generativeai)"
        "\n2. Defines solve_packing(llm_function, n=26) -> list"
        "\n3. Has __main__ block that configures Gemini and outputs JSON"
    ),
    config=GEPAConfig(
```

Mathematical Formulation

$$\max_{x_i, y_i, r_i} \sum_{i=1}^{26} r_i$$

Subject to containment for each circle:

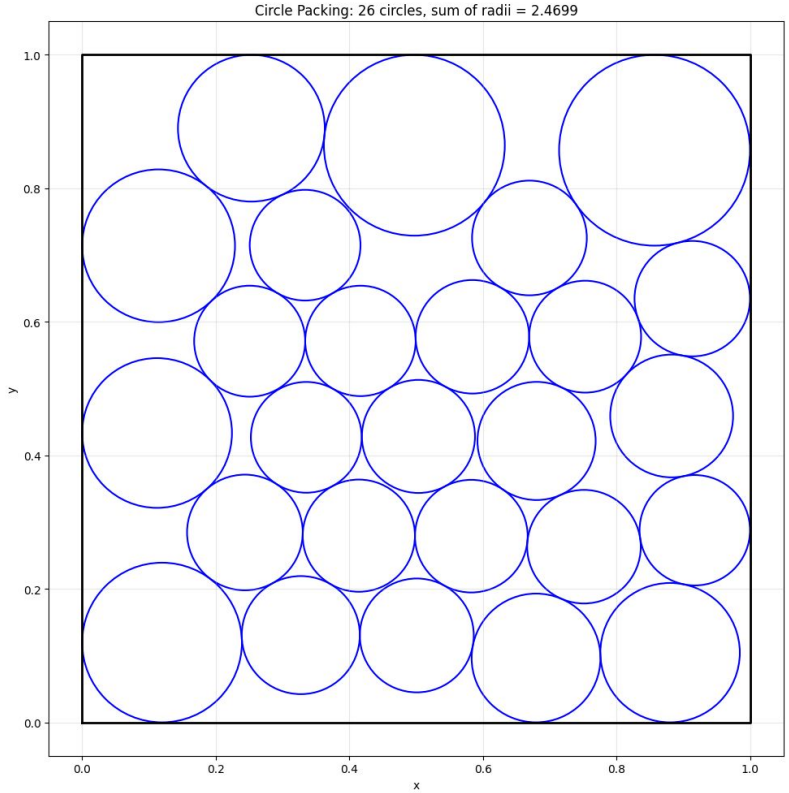
$$r_i \leq x_i \leq 1 - r_i, \quad r_i \leq y_i \leq 1 - r_i, \quad r_i \geq 0$$

and non-overlap for all i ≠ j:

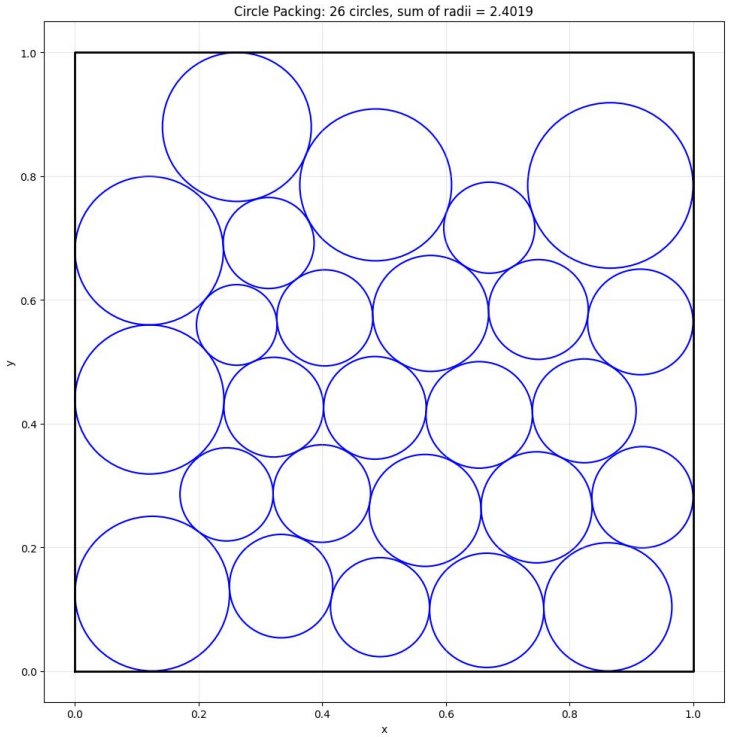
$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq r_i + r_j.$$

Initial Results (n=20)

Feeding no information

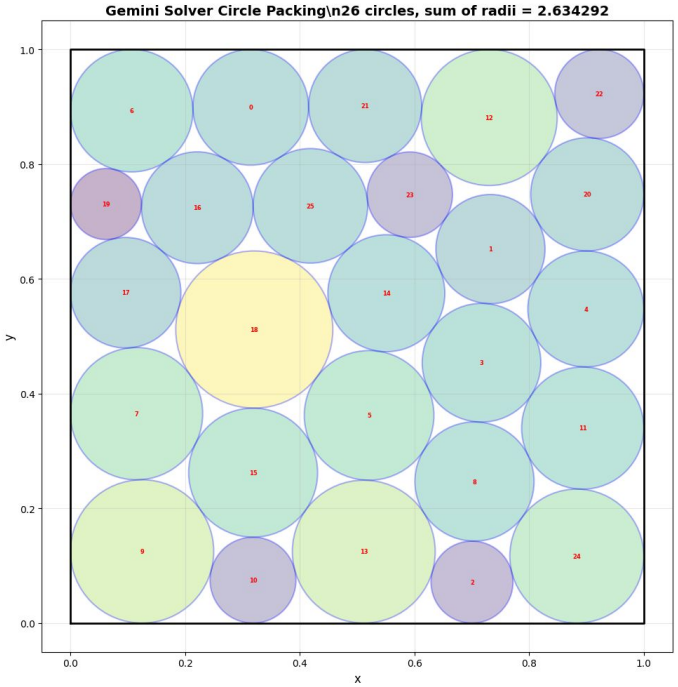


Feeding false information: told best solution was greater than it actually was



o3-mini

Gemini 2.5 Flash: Feeding state of the art solution (numerical value)



Next steps

- Reduce invalid candidates via stronger contract prompting and minimal self-checks inside the solver
- History-aware GEPA: archive prompts/programs to avoid missing good branches and to prevent rediscovery
- Two-model setup: cheaper model for reflection and a stronger model for solver calls (or vice versa)